

Teoria de POO. Parte B: herança e polimorfismo

Esta parte apresenta conteúdo novo. Ela não exige decorar a Parte A. Ela exige uma coisa: você reconhece que cada personagem guarda o próprio estado e executa as próprias ações. Se isso está claro, comece por aqui.

Hoje o RPG tem um limite. Todos os combatentes são a mesma classe `Personagem`. `Goblin`, `Orc`, `Troll` e o jogador mudam só nos números. Isso funcionou até agora. A Parte B mostra o próximo passo. Ela mostra o que fazer quando os personagens precisam ser diferentes no comportamento, não só nos números.

1. O problema

Leia esta seção antes de qualquer sintaxe nova.

O RPG vai ganhar tipos de herói: `Guerreiro`, `Mago` e `Ladino`.

- O `Guerreiro` tem muita vida. Ele ataca corpo a corpo.
- O `Mago` tem pouca vida. O ataque dele é mágico e ignora parte da defesa do alvo.
- O `Ladino` tem vida mediana. O ataque dele tem chance de causar dano dobrado.

Hoje existe só a classe `Personagem`. Para separar os tipos, você faria algo assim:

```
class Personagem:
    def __init__(self, nome, vida, ataque, defesa, pocoes, tipo):
        self.tipo = tipo
        ...

    def atacar(self, alvo):
        if self.tipo == "mago":
            # ataque mágico, ignora parte da defesa
            ...
        elif self.tipo == "guerreiro":
            # ataque físico normal
            ...
        elif self.tipo == "ladino":
            # ataque com chance de crítico
            ...
```

Agora pense em cinco tipos. Depois em oito. Cada tipo novo obriga você a abrir a classe `Personagem`. Cada tipo novo adiciona mais um `elif` dentro de `atacar`. Depois dentro de `usar_pocao`. Depois dentro de tudo. Um método simples vira uma escada de `if/elif`. A escada cresce sem parar. Um erro em um tipo pode quebrar os outros.

Existe um segundo problema, além da escada. Cada tipo novo também obriga o programa a lembrar o nome exato da string, `"guerreiro"`, `"mago"`, `"ladino"`. Um erro de digitação, como `"guerreio"`, não gera erro nenhum. O `elif` simplesmente não bate com nenhum caso e o personagem não ataca. O programa erra em silêncio.

O problema é claro. O que é comum a todos e o que é específico de cada tipo fica no mesmo lugar. Herança separa essas duas coisas.

Perguntas

1. No código acima, o que é igual para `Guerreiro`, `Mago` e `Ladino`? O que é diferente?
2. Você adiciona um quarto tipo. Quantos métodos com `if self.tipo == ...` você precisa

abrir? Por que isso é frágil?

3. Descreva em uma frase onde você quer escrever o ataque do Mago, sem tocar no ataque do Guerreiro.
 4. Por que um erro de digitação na string do tipo, como "guerreio" em vez de "guerreiro", é um problema difícil de encontrar?
-

2. Herança

Herança cria uma classe a partir de outra. A nova classe aproveita tudo da outra. A nova classe adiciona ou troca só o que é diferente.

Use estes nomes:

- a classe aproveitada é a classe-mãe. Aqui: Personagem .
- a classe que aproveita é a classe-filha. Aqui: Guerreiro , Mago e Ladino .

Escreva o nome da mãe entre parênteses:

```
class Guerreiro(Personagem):  
    pass  
  
class Mago(Personagem):  
    pass
```

Só com isso, um Guerreiro já tem vida , receber_dano , defender , esta_vivo e mostrar_status . Ele herda tudo de Personagem . Você não reescreve nada:

```
heroi = Guerreiro("Thoric", 120, 15, 6, 2)  
heroi.mostrar_status()      # método herdado de Personagem  
heroi.receber_dano(10)      # método herdado de Personagem  
print(heroi.esta_vivo())    # método herdado de Personagem
```

Repare no que aconteceu. Você não escreveu receber_dano dentro de Guerreiro . O Python procura o método na classe Guerreiro . Não encontra. Sobe para a classe-mãe, Personagem . Encontra lá e executa. Esse é o mecanismo por trás da herança: o Python sempre procura primeiro na classe do objeto, e só sobe para a mãe quando não encontra.

Guarde a regra central. O que é comum fica na mãe. O que é específico fica na filha. Uma regra compartilhada, como "a vida não fica negativa", existe em um único lugar. Ela vale para todos os tipos de forma automática.

Nota. Uma classe-filha pode ter mais de uma classe-filha própria depois dela. Este material não usa esse caso. Aqui, Personagem é sempre a mãe direta de Guerreiro , Mago e Ladino .

Perguntas

1. Depois de `class Mago(Personagem): pass` , quais métodos um objeto Mago pode chamar? De onde vêm esses métodos?
2. Você corrige um erro dentro de `receber_dano()` na classe Personagem . O Guerreiro e o Mago também ficam corrigidos? Justifique.
3. Sobe ou desce. Diga se cada item fica na mãe Personagem ou em uma filha: vida ; defender ; mana ; a forma de atacar do Mago; esta_vivo .
4. O Python procura um método primeiro na classe do objeto ou primeiro na classe-mãe? O que acontece quando ele não encontra o método em nenhuma das duas?
5. Cenário novo. Você cria `class Ladino(Personagem): pass` e chama

ladino.atacar(inimigo) . Qual atacar roda? De onde ele vem?

3. Reaproveitar o construtor com super().__init__()

O Guerreiro, o Mago e o Ladino ainda precisam de nome, vida, ataque, defesa e pocoes. O `__init__` da mãe já monta esses atributos. Você não reescreve esse construtor. Use `super()` para chamar o construtor da classe-mãe:

```
class Guerreiro(Personagem):
    def __init__(self, nome):
        super().__init__(nome, 120, 15, 6, 2)

class Mago(Personagem):
    def __init__(self, nome):
        super().__init__(nome, 80, 20, 3, 4)
        self.mana = 30
```

Veja o que acontece:

- `super().__init__(...)` executa o `__init__` de `Personagem`. Ele monta nome, vida, defesa e os outros atributos.
- cada filha fixa os próprios números. O Guerreiro nasce com 120 de vida. O Mago nasce com 80.
- o Mago adiciona um atributo próprio: `self.mana`.

Agora você cria heróis com poucas linhas. Cada tipo já vem com os próprios valores:

```
heroi = Guerreiro("Thoric")
mago = Mago("Eldrin")
print(heroi.vida) # 120
print(mago.vida) # 80
print(mago.mana) # 30
```

Repare na ordem das linhas dentro do `__init__` do Mago. `super().__init__(...)` vem primeiro. `self.mana = 30` vem depois. Essa ordem importa. `super().__init__(...)` cria o objeto com nome, vida, ataque, defesa e pocoes. Só depois disso o Mago acrescenta o próprio atributo. Se você inverter a ordem, o Python ainda funciona, porque `self.mana = 30` não depende nada da mãe. Mas o hábito de chamar `super().__init__()` primeiro evita erros quando um atributo novo depende de um atributo antigo.

Agora aplique a mesma ideia ao Ladino. O Ladino tem vida mediana, ataque considerável, defesa baixa e não tem nenhum atributo extra além dos cinco de `Personagem`:

```
class Ladino(Personagem):
    def __init__(self, nome):
        super().__init__(nome, 90, 18, 4, 3)
```

Perguntas

1. Explique a linha `super().__init__(nome, 80, 20, 3, 4)` passo a passo. Quais atributos o Mago tem logo depois dela?
2. O que falta. Alguém escreveu a classe Mago assim:

```
class Mago(Personagem):
    def __init__(self, nome):
        self.mana = 30
```

- O que dá errado quando o jogo chama `Mago("Eldrin").mostrar_status()` ? Qual linha falta?
3. O Guerreiro não define `self.mana` . O Mago define. Onde fica um atributo que só um tipo usa?
 4. O Ladino, como escrito acima, não tem nenhum atributo além dos cinco herdados. Ele ainda precisa da linha `super().__init__(...)` ? Justifique.
 5. Cenário novo. Um Arqueiro nasce com 100 de vida, 16 de ataque, 5 de defesa, 3 poções e um atributo próprio, `flechas` , que começa em 10. Escreva o `__init__` completo do Arqueiro.
 6. O que falta. Alguém escreveu `super().__init__(nome, 90, 18, 4, 3)` , sem os parênteses depois de `super()` . Qual é o erro que o Python mostra?
-

4. Sobrescrita: a filha muda um comportamento

Herança não obriga a filha a manter tudo igual. A filha pode definir um método com o mesmo nome de um método da mãe. Então a versão da filha vale para os objetos daquele tipo. Isso se chama sobrescrita.

O Guerreiro usa o `atacar` herdado, normal. O Mago sobreescreve `atacar` . O ataque do Mago é mágico e ignora parte da defesa do alvo:

```
class Mago(Personagem):
    def __init__(self, nome):
        super().__init__(nome, 80, 20, 3, 4)
        self.mana = 30

    def atacar(self, alvo):
        dano = random.randint(self.ataque - 2, self.ataque + 5)
        dano_magico = dano + alvo.defesa # compensa a defesa: a magia a ignora
        alvo.receber_dano(dano_magico)
        print(self.nome, "lançou uma magia de", dano, "de dano")
```

Agora atacar significa coisas diferentes por tipo:

```
guerreiro.atacar(inimigo) # usa o atacar herdado de Personagem
mago.atacar(inimigo)      # usa o atacar próprio do Mago
```

Compare com a Seção 1. Aqui não existe `if self.tipo == "mago"` . O comportamento do Mago fica todo dentro da classe Mago . O comportamento do Guerreiro não muda.

A sobrescrita não é exclusiva de `atacar` . Qualquer método herdado pode ser sobrescrito. Veja o Ladino, que sobreescreve `atacar` de um jeito diferente do Mago, com uma chance de dano dobrado:

```

class Ladino(Personagem):
    def __init__(self, nome):
        super().__init__(nome, 90, 18, 4, 3)

    def atacar(self, alvo):
        rolagem = random.randint(1, 20)
        if rolagem < 5:
            print(self.nome, "errou o ataque")
            return
        dano = random.randint(self.ataque - 3, self.ataque + 3)
        critico = random.randint(1, 100) <= 25
        if critico:
            dano = dano * 2
            print(self.nome, "acertou um golpe crítico")
        alvo.receber_dano(dano)
        print(self.nome, "causou", dano, "de dano")

```

Repare que o Ladino também sobrescreve `atacar`, mas a lógica dele é totalmente diferente da lógica do Mago. Cada classe-filha decide a própria versão. Uma não precisa saber da outra.

Erros comuns na sobrescrita

Um erro comum é esquecer `self` no cabeçalho do método sobrescrito. Sem `self`, o método não recebe o próprio objeto como primeiro parâmetro, e o Python falha na chamada.

Outro erro comum é mudar o nome do método por engano, como escrever `def atacar_mago(self, alvo):` em vez de `def atacar(self, alvo):`. Nesse caso não existe sobrescrita nenhuma. A classe Mago ganha um método novo, `atacar_mago`, e continua usando o `atacar` herdado de `Personagem`, sem nenhum aviso de erro.

Um terceiro erro é esquecer que a assinatura da filha deve aceitar os mesmos parâmetros que o chamador espera. Se `combate()` sempre chama `atacar(alvo)` com um argumento, um `atacar(self, alvo, bonus)` na filha, com um parâmetro obrigatório a mais, quebra a chamada.

Perguntas

1. Qual `atacar` roda em `guerreiro.atacar(inimigo)`? E em `mago.atacar(inimigo)`? Como o Python decide?
2. O `atacar` do Mago ainda chama `alvo.receber_dano(...)`. Por que isso importa? Use a ideia de que o alvo cuida da própria vida.
3. Cenário novo. Um Curandeiro sobrescreve `defender` para também recuperar 5 de vida além de preparar a defesa. Escreva o método `defender` do Curandeiro.
4. O que falta. Alguém escreveu `def Atacar(self, alvo):`, com A maiúsculo, dentro da classe Mago. O que acontece quando o jogo chama `mago.atacar(inimigo)`? O ataque mágico roda?
5. O que falta. Alguém escreveu `def atacar(self, alvo, bonus):` na classe Ladino, com um terceiro parâmetro obrigatório. O laço de combate chama `atacar(alvo)`, com um argumento só. O que o Python mostra?
6. Cenário novo. O Ladino, além da chance de crítico, ganha uma segunda regra: se errar o ataque, ele não gasta o turno de defesa do inimigo. Isso muda o método `atacar` do Ladino, o método `atacar` do Guerreiro, os dois, ou nenhum? Justifique.

5. Estender o comportamento da mãe com `super()`

A Seção 4 mostrou sobrescrita que troca o comportamento por completo. Existe uma segunda

forma de sobrescrita: a filha aproveita o comportamento da mãe e acrescenta algo a mais. Para isso, a filha chama o método da mãe dentro do próprio método, com `super()`.

Você já usa `super()` para o construtor, na Seção 3. A mesma ideia funciona para qualquer método, não só para `__init__`.

Veja um Paladino que sobrescreve `defender`. Ele mantém o comportamento normal de defesa e acrescenta uma cura pequena:

```
class Paladino(Personagem):
    def __init__(self, nome):
        super().__init__(nome, 100, 12, 8, 3)

    def defender(self):
        super().defender()
        cura = 5
        self.vida = min(self.vida + cura, self.vida_maxima)
        print(self.nome, "reza enquanto defende e recupera", cura, "de vida")
```

Passo a passo:

- `super().defender()` executa o `defender` de `Personagem`. Ele faz `self.defendendo = True` e mostra a mensagem original.
- depois disso, o método do Paladino roda o próprio código: cura 5 de vida e mostra uma segunda mensagem.

Compare com a Seção 4. Lá, o atacar do Mago não chama `super().atacar()`. Ele substitui o comportamento inteiro, porque o ataque mágico não tem nada em comum com o ataque físico. Aqui, o `defender` do Paladino chama `super().defender()`, porque o comportamento de `defender` é o mesmo, e o Paladino só acrescenta um efeito a mais em cima dele.

Nota. Use `super()` dentro de um método sobrescrito quando a filha quer o comportamento da mãe e mais alguma coisa. Não use `super()` quando a filha quer um comportamento totalmente diferente, como o atacar do Mago.

Perguntas

1. O que `super().defender()` faz, exatamente, dentro do método `defender` do Paladino?
2. Se você apagar a linha `super().defender()` do Paladino, o atributo `self.defendendo` ainda vira `True` quando o Paladino defende? Justifique.
3. Cenário novo. O Guerreiro sobrescreve `usar_pocao` para, além de curar, remover qualquer efeito de veneno. Ele deve chamar `super().usar_pocao()`? Justifique com a regra desta seção.
4. Compare o atacar do Mago, na Seção 4, com o `defender` do Paladino, nesta seção. Qual dos dois substitui o comportamento da mãe e qual dos dois estende o comportamento da mãe?

6. Polimorfismo

Veja o laço de batalha que você já escreveu:

```
def combate(jogador, inimigo):
    while jogador.esta_vivo() and inimigo.esta_vivo():
        ...
        turno_do_jogador(jogador, inimigo)
        if inimigo.esta_vivo():
            inimigo.atacar(jogador)
        ...
```

A linha `inimigo.atacar(jogador)` não muda. O inimigo pode ser um Goblin, um Mago ou um Ladino. Cada objeto responde à mesma chamada do próprio jeito. O Mago lança magia. O Guerreiro golpeia. O Ladino tenta o crítico. Quem escreve o laço não precisa saber o tipo.

Isso se chama polimorfismo. Uma mesma chamada, `atacar(...)`, produz comportamentos diferentes. O comportamento dependendo objeto que recebe a chamada.

Um combate passo a passo

Imagine esta lista de inimigos:

```
inimigos = [Goblin("Grunt", 40, 8, 2, 0), Mago("Sombrio"), Ladino("Faca")]

for inimigo in inimigos:
    combate(jogador, inimigo)
```

O laço `for` percorre a lista. Dentro de `combate()`, a linha `inimigo.atacar(jogador)` roda uma vez por turno. Na primeira luta, `inimigo` é um Goblin, e o `atacar` herdado de `Personagem` roda. Na segunda luta, `inimigo` é um Mago, e o `atacar` sobrescrito do Mago roda, com dano mágico. Na terceira luta, `inimigo` é um Ladino, e o `atacar` sobrescrito do Ladino roda, com chance de crítico.

O código de `combate()` é sempre o mesmo, em todas as três lutas. Nenhuma linha dentro de `combate()` menciona Goblin, Mago ou Ladino pelo nome. O laço só sabe que recebe um `Personagem`, e chama `atacar`. O objeto certo faz a ação certa sozinho.

Compare os dois desenhos:

- sem herança: um `atacar` grande com `if self.tipo == ...` para cada caso. Cada tipo novo abre esse método. Cada tipo novo arrisca quebrar os outros.
- com herança e polimorfismo: cada tipo tem o próprio `atacar` na própria classe. O laço chama `inimigo.atacar(jogador)`. O objeto certo faz a ação certa. Um tipo novo é uma classe nova. O laço não muda.

Assim, adicionar comportamento vira adicionar uma classe. Você não edita as classes antigas.

Perguntas

1. Explique com suas palavras por que `inimigo.atacar(jogador)` funciona para um Mago, um Ladino e um Guerreiro, sem nenhum `if` no laço.
2. Você adiciona a classe `Arqueiro` e coloca ela na lista de inimigos. Quantas linhas de `combate()` mudam? Justifique.
3. Compare os dois desenhos da Seção 1 e da Seção 6 para adicionar um tipo novo. Em qual desenho o código antigo corre risco de quebrar? Justifique.
4. Ligue o conceito ao nome. Diga qual pilar cada trecho representa, abstração, encapsulamento, herança ou polimorfismo:
 - `class Mago(Personagem):`
 - `alvo.receber_dano(dano_magico)`
 - `inimigo.atacar(jogador)` chamando versões diferentes por tipo
 - `esta_vivo()` escondendo `self.vida > 0`
5. Na lista `inimigos = [Goblin(...), Mago(...), Ladino(...)]`, o laço `for inimigo in`

`inimigos`: precisa verificar o tipo de cada elemento antes de chamar `inimigo.atacar(jogador)` ? Justifique com a ideia de polimorfismo.

7. Erros comuns ao herdar

Esta seção reúne, em um só lugar, os erros mais frequentes ao criar uma classe-filha. Use como referência rápida.

Esquecer `super().__init__()`. A filha define `__init__` próprio, mas não chama o construtor da mãe. Nenhum dos atributos herdados existe, e qualquer método herdado que use `self.vida` ou `self.nome` falha.

Passar os parâmetros errados para `super().__init__()`. A ordem dos números importa. O `__init__` de `Personagem` espera `nome, vida, ataque, defesa, pocoes`, nessa ordem. Trocar `vida` e `ataque` de posição cria um personagem com números invertidos, sem nenhum erro visível.

Sobrescrever um método com um nome parecido, mas não igual. `def Atacar` em vez de `def atacar`, ou `def atacar_do_mago` em vez de `def atacar`. O Python não avisa. A classe simplesmente ganha um método novo e continua usando o método herdado.

Mudar a assinatura do método sobrescrito. O método da mãe espera `atacar(self, alvo)`. Se a filha escrever `atacar(self, alvo, critico)`, com um parâmetro a mais sem valor padrão, qualquer chamada `objeto.atacar(outro)` feita pelo resto do programa quebra.

Esquecer `self`. Um método sobrescrito sem `self` no cabeçalho não funciona como método de instância. O Python aponta um erro na hora da chamada.

Colocar lógica específica de um tipo na classe-mãe. Se só o Mago tem `mana`, o atributo `mana` não deve entrar no `__init__` de `Personagem`. Isso obrigaria `Guerreiro` e `Ladino` a carregar um atributo que nunca usam.

Perguntas

1. Um `Guerreiro` nasce sem `vida`, sem `ataque` e sem `defesa`, e qualquer chamada a `mostrar_status()` gera erro. Qual dos erros desta seção provavelmente aconteceu?
 2. Um Mago é criado com `super().__init__(nome, 20, 80, 3, 4)`, com `vida` e `ataque` trocados de posição. O objeto existe e não gera erro imediato. Qual sintoma aparece mais tarde, durante o jogo?
 3. Você escreve `def atacar(self, alvo, bonus_critico):` na classe `Ladino`, sem valor padrão para `bonus_critico`. O laço de combate chama `inimigo.atacar(jogador)`. O que o Python mostra?
 4. Por que colocar `self.mana = 30` dentro do `__init__` de `Personagem`, em vez de dentro do `__init__` do Mago, é um erro de projeto, mesmo que o código rode sem erro?
-

8. Quando não usar herança

Use herança quando os tipos são variações da mesma coisa. Todos são personagens. Todos atacam, defendem e têm vida. `Guerreiro`, `Mago` e `Ladino` passam no teste: cada um é um `Personagem`.

Não use herança quando a relação é "tem um", e não "é um". Um personagem tem uma mochila. Uma mochila não é um personagem. Nesse caso a mochila é outro objeto, guardado como atributo. Herança forçada, onde a relação é "tem um", cria hierarquias confusas.

Guarde uma regra prática para o RPG. Se a frase "X é um Personagem" for verdadeira, X pode ser uma classe-filha. Se a frase for "X tem um Y", então Y é um atributo, não uma classe-mãe. Veja um exemplo do erro. Alguém tenta modelar o inventário como uma classe-filha de Personagem :

```
class Inventario(Personagem): # errado: um Inventário não é um Personagem
    def __init__(self, capacidade):
        self.capacidade = capacidade
        self.itens = []
```

Um Inventário não ataca. Um Inventário não tem vida. A frase "um Inventário é um Personagem" é falsa. O jeito certo é o Inventário existir como objeto separado, e o Personagem guardar uma referência a ele:

```
class Inventario:
    def __init__(self, capacidade):
        self.capacidade = capacidade
        self.itens = []

class Personagem:
    def __init__(self, nome, vida, ataque, defesa, pocoes):
        ...
        self.inventario = Inventario(10) # o Personagem tem um Inventario
```

Esse segundodesenhofica para a Parte C, que trata desse tipo de relação com mais detalhe.

Perguntas

1. Diga se cada par é "é um" (herança) ou "tem um" (atributo): Mago e Personagem; Personagem e Inventário; Chefe e Personagem; Personagem e Arma.
2. Por que `class Inventario(Personagem):` é um erro de projeto, mesmo que o código rode sem falhar?
3. O jogo vai ganhar uma Arma com atributos próprios, como `dano_bonus` e `nome_arma`. A Arma herda de Personagem ou fica como atributo do personagem? Use a regra desta seção.
4. Releia o problema da Seção 1 e o resultado da Seção 6. Em uma frase, o que a herança resolveu?

9. Estudo de caso: um combate completo com três tipos

Esta seção junta tudo. Ela mostra o código final de três classes-filhas e o resultado de um combate entre elas, turno a turno.

```
import random

class Personagem:
    def __init__(self, nome, vida, ataque, defesa, pocoes):
        self.nome = nome
        self.vida = vida
        self.vida_maxima = vida
        self.ataque = ataque
        self.defesa = defesa
        self.pocoes = pocoes
        self.defendendo = False
```

```

def mostrar_status(self):
    print("\n", self.nome)
    print("Vida:", self.vida, "/", self.vida_maxima)
    print("Poções:", self.pocoes)

def esta_vivo(self):
    return self.vida > 0

def receber_dano(self, quantidade):
    dano_real = quantidade - self.defesa
    if self.defendendo:
        dano_real = dano_real // 2
        self.defendendo = False
    dano_real = max(0, dano_real)
    self.vida = max(0, self.vida - dano_real)

def atacar(self, alvo):
    rolagem = random.randint(1, 20)
    if rolagem < 5:
        print(self.nome, "errou o ataque")
        return
    dano = random.randint(self.ataque - 3, self.ataque + 3)
    alvo.receber_dano(dano)
    print(self.nome, "causou", dano, "de dano")

def defender(self):
    self.defendendo = True
    print(self.nome, "preparou a defesa")

def usar_pocao(self):
    if self.pocoes == 0:
        print("Sem poções")
        return
    if self.vida == self.vida_maxima:
        print("Vida cheia")
        return
    self.vida = min(self.vida + 25, self.vida_maxima)
    self.pocoes -= 1
    print(self.nome, "usou uma poção")

```

```

class Guerreiro(Personagem):
    def __init__(self, nome):
        super().__init__(nome, 120, 15, 6, 2)

```

```

class Mago(Personagem):
    def __init__(self, nome):
        super().__init__(nome, 80, 20, 3, 4)
        self mana = 30

    def atacar(self, alvo):
        dano = random.randint(self.ataque - 2, self.ataque + 5)
        dano_magico = dano + alvo.defesa
        alvo.receber_dano(dano_magico)
        print(self.nome, "lançou uma magia de", dano, "de dano")

```

```

class Ladino(Personagem):
    def __init__(self, nome):

```

```

super().__init__(nome, 90, 18, 4, 3)

def atacar(self, alvo):
    rolagem = random.randint(1, 20)
    if rolagem < 5:
        print(self.nome, "errou o ataque")
        return
    dano = random.randint(self.ataque - 3, self.ataque + 3)
    critico = random.randint(1, 100) <= 25
    if critico:
        dano = dano * 2
        print(self.nome, "acertou um golpe crítico")
    alvo.receber_dano(dano)
    print(self.nome, "causou", dano, "de dano")

```

Agora imagine este trecho de `main.py` :

```

heroi = Guerreiro("Thoric")
adversarios = [Mago("Sombrio"), Ladino("Faca")]

for inimigo in adversarios:
    combate(heroi, inimigo)

```

Na primeira luta, `inimigo` é um `Mago`. A linha `inimigo.atacar(heroi)`, dentro de `combate()`, roda o `atacar` sobrescrito do `Mago`. O dano soma a `alvo.defesa`, então a defesa do `Guerreiro` não reduz o dano da magia.

Na segunda luta, `inimigo` é um `Ladino`. A mesma linha `inimigo.atacar(heroi)` agora roda o `atacar` sobrescrito do `Ladino`. Existe uma chance de 25% do dano dobrar.

Repare de novo: `combate()` não muda entre as duas lutas. Nenhuma linha dele sabe se o inimigo é `Mago` ou `Ladino`. Essa é a demonstração final do polimorfismo: o mesmo laço, escrito uma vez, funciona para qualquer classe-filha de `Personagem` que exista hoje, e para qualquer classe-filha que você criar amanhã.

Perguntas

1. No trecho de `main.py` acima, o `heroi` é sempre um `Guerreiro`. O `atacar` do `Guerreiro` está sobrescrito ou herdado?
2. Se você trocar a ordem da lista para `[Ladino("Faca"), Mago("Sombrio")]`, alguma linha de `combate()` precisa mudar? Justifique.
3. O `Mago` tem `self.mana`, mas o `atacar` do `Mago`, neste código, nunca usa `self.mana`. Isso é um erro de projeto? O que falta para o atributo `mana` fazer sentido no jogo?
4. Adicione um quarto tipo à lista de adversários, um `Arqueiro` com o `atacar` herdado de `Personagem`, sem sobrescrita. Ele precisa de alguma mudança em `combate()`? Justifique com o que você aprendeu na Seção 6.

Perguntas finais

1. Em uma frase, qual é a diferença entre herança e polimorfismo?
2. Um novo tipo de personagem só muda números, não comportamento. Ele precisa sobrescrever algum método? Justifique.
3. Cite um método que faz sentido sobrescrever com `super()`, no espírito da Seção 5, e um método que faz sentido sobrescrever por completo, no espírito da Seção 4. Explique a diferença entre os dois casos.

4. Reveja a lista de erros da Seção 7. Qual desses erros você já cometeu, ou acha mais fácil de cometer, no seu próprio código? Por quê?
 5. O Inventário da Seção 8 não herda de Personagem. Explique, com suas palavras, por que essa decisão está certa.
-

Sequência de estudo

1. Leia a Seção 1 inteira antes de ver sintaxe nova. Entenda o problema.
2. Escreva você mesmo as classes Guerreiro, Mago e Ladino. Use `super().__init__()`.
3. Crie um objeto de cada tipo. Chame um método herdado, como `mostrar_status()`.
4. Depois escreva o atacar sobrescrito do Mago e do Ladino. Teste os tipos lado a lado.
5. Escreva o defender do Paladino da Seção 5, com `super().defender()`. Confirme que ele mantém o comportamento original e acrescenta a cura.
6. Rode o laço de batalha sem mudar o laço. Confirme que Mago, Guerreiro e Ladino lutam de forma diferente. Esse é o momento em que você vê o polimorfismo.
7. Revise a lista de erros da Seção 7. Confira, um por um, se algum deles aparece no seu próprio código.
8. Depois disso, o RPG aceita tipos novos sem reescrever o laço.